有监督微调

第 5 章

第 7 课时

微调概览 · 参数高效微调 · LoRA · QLoRA

著者
邓杭　杨耀东

教材
《大语言模型：训练、精调与对齐》

出版
清华大学出版社



本章目录

第 7 课时

5.1 **语言模型微调概览** — 微调类型与 PEFT 概览 — 第 3 页

5.2 **参数高效微调** — 软提示、前缀、适配器、LoRA — 第 23 页

5.3 **LoRA 算法原理** — 低秩重参数化与子空间分析 — 第 32 页

5.4 **QLoRA** — LoRA 上量化基座，降低显存 — 第 49 页

5.5 **微调方法对比** — 三种微调的精度、显存与吞吐 — 第 52 页

5.6 **小结** — 回顾与 LoRA 的扩展方向 — 第 56 页

阅读建议 先理解微调与预训练的差别（5.1），再读 PEFT 与 LoRA 的原理与实现（5.2–5.4）。

5.1

语言模型微调概览

预训练给出通用能力，微调把这种能力导向具体任务；本节先分清微调的类型与代价，再落到有监督微调的数据与目标。

本节内容

5.1.1 介绍

5.1.2 有监督微调

5.1.3 提示词微调

5.1.4 微调中的灾难性遗忘

微调的定义

预训练语言模型从海量文本中学到鲁棒的语言表征与丰富的世界知识；微调（Fine-Tuning）则是在已训练好的模型基础上，用特定任务或领域的数据继续调整参数，把通用能力导向具体任务。

预训练

自监督目标，在通用语料上学语言的一般规律与世界知识；标签由序列自身位移得到。

产出的是一台“会语言”的基座模型。

微调

在基座模型上，用任务数据继续训练；数据是成对的输入与期望输出。

产出的是一台“会做某件事”的模型。

- 本章讨论**有监督微调**（SFT）；对齐到人类偏好或规则奖励的强化学习类方法见第 8 章。
- 微调赋予模型的关键能力之一是**指令跟随**：理解自然语言形式的请求，并生成符合要求的响应。

预训练模型适合微调的原因

传统监督模型迁移到新任务时，往往要改网络结构、设计复杂流程，并收集大量任务标注；预训练语言模型则可以直接用“文本进、文本出”的方式交互。

优势来源一

文本的通用性

指令、输入与答案都能写成同一种词元序列，不必为每个任务另造输入接口。

优势来源二

强零样本能力

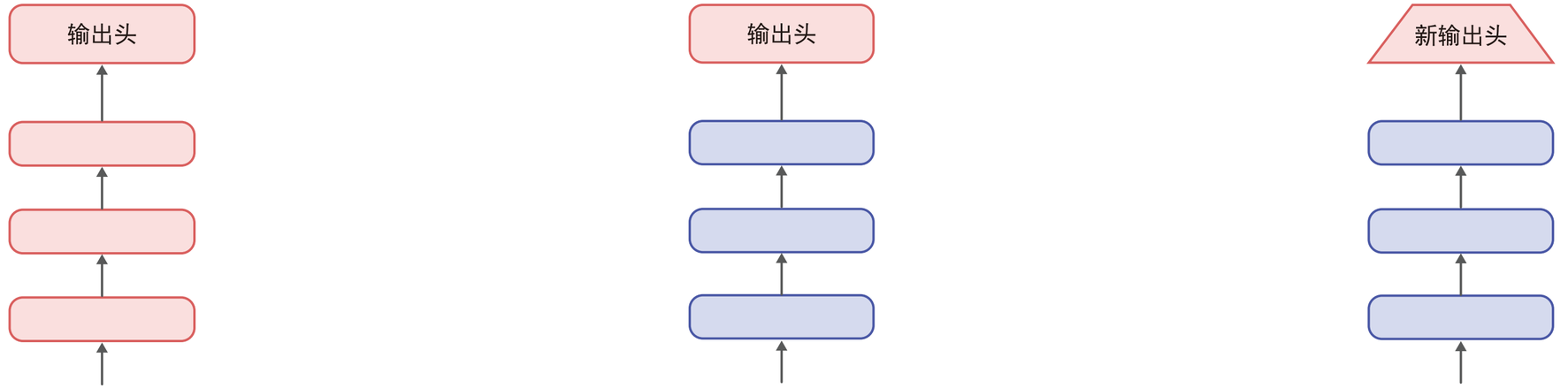
基座模型已具备语言表征与推理能力，少量样本即可激活任务能力。

- **微调的核心要素是数据与训练方法**：数据组织成（输入指令/问题，期望输出/答案）的监督样本对，可来自通用指令集或特定任务集。
- 全参微调成本高昂，**参数高效微调**（PEFT）随之出现：只调整少量参数，力求接近全参微调的效果。

延伸：多模态场景中，把视觉表征投影对齐到语言模型的输入嵌入空间再做指令微调，模型即可处理图文混合输入。

按参数更新范围划分的微调方法

根据参数更新范围的不同，微调可分为三类：红色块参数可更新，蓝色块参数冻结。



(a) 全参数微调　(b) 部分层微调　(c) 输出层调整

图 5.1　传统微调方法对比图

- 三类方法更新范围依次缩小：全部权重 → 特定层（如输出头）→ 仅输出层结构。
- 缩小更新范围能省资源，但传统做法性能一般，这正是后续参数高效微调的出发点。

基于适配器的微调

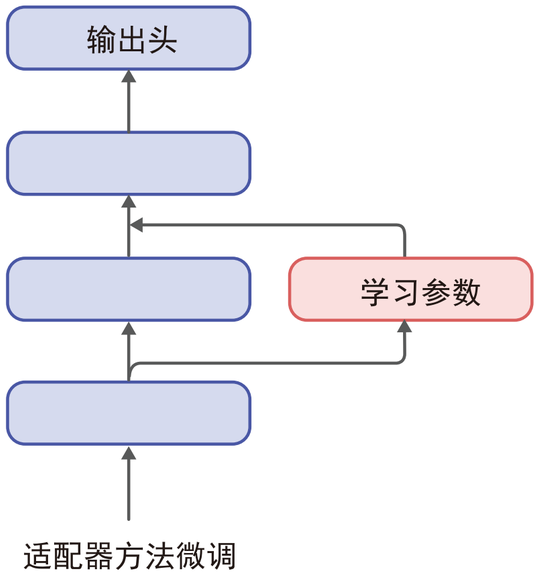


图 5.2　适配器微调模式

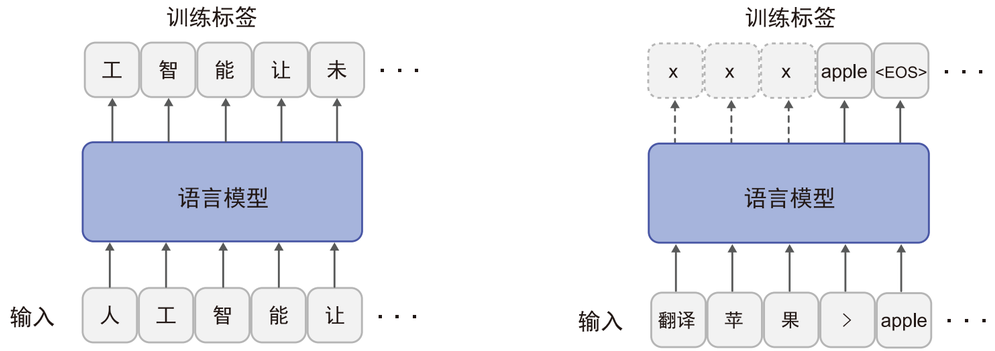
适配器（Adapter）是一种轻量级可学习模块，插入到模型结构中，只训练少量额外参数。

- 原模型参数保持冻结，新增的适配器模块承担任务适配。
- 这类“只训练少量额外参数”的方法统称为**参数高效微调**。
- 与部分层微调的区别：不改动原有层的取舍，而是在层间**插入**新模块。

适配器方法将在 5.2 节展开，并在 5.3 节与 LoRA 对照。

微调标签的来源

预训练不涉及特定下游任务，标签由序列自身位移自动生成；有监督微调的输入-输出对需要明确定义，输出通常由人工标注。



(a) 预训练　　(b) 有监督微调

图 5.3　预训练和微调对比图

- **预训练**：给定输入序列“人工智能让未来更加美好…”，标签即为左移一位的“工智能让未来更加美好…”。
- **有监督微调**：机器翻译、对话、摘要、分类等任务，需要成对的输入与期望输出。

预训练与微调的差别

预训练数据无需特定格式，可涵盖代码、数学证明与诗歌；微调数据要格式化成（指令-回答）对，且只计算回答词元的损失。

表 5.1　预训练与有监督微调的比较

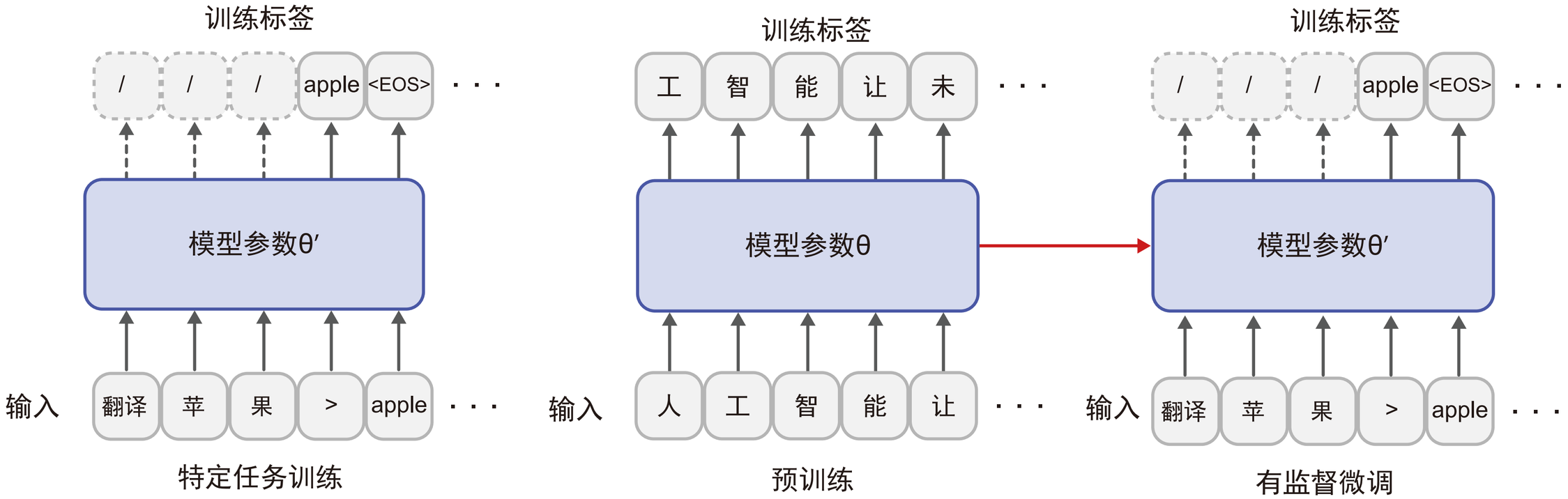
特征	预训练	微调
数据规模	海量通用数据	较小规模的任务特定数据，如万条数据
数据格式	无	有特定的模板
计算资源	需要大量计算资源	较少的计算资源
训练目标	学习语言的一般特征和表示，不适配特定任务	适应特定任务或领域
参数更新	从随机初始化开始训练所有参数	在预训练参数基础上小幅调整
成本	极高	低

以翻译为例：数据格式化为“翻译苹果为英文>apple”，拼接后送入模型前向计算，仅对回答词元的标签计损失。

依据 《大语言模型：训练、精调与对齐》表 5.1；第 5.1.1 节

从头训练与预训练微调

解决同一个特定任务有两条路径：用任务数据在随机参数模型上从头训练，或先在大规模语料上预训练、再用任务数据微调。



(a) 从头开始训练：需大量任务相关数据

(b) 预训练微调：少量数据即可，效率与泛化更好

图 5.4　特定任务训练和基于预训练模型微调训练范式对比

两种训练范式的对比

预训练微调在效率、数据需求、性能、泛化与稳定性上都占优，代价是需要先完成一次昂贵的预训练。

表 5.2　特征任务学习训练范式特性对比

特征	从头训练	基于预训练模型微调
训练效率	低效，需要从零开始学习	高效，利用已有知识
数据需求	需要大量任务相关数据	只需少量任务数据
性能表现	受数据量限制	通常能达到更好的效果
泛化能力	较差	较强
训练稳定性	不稳定	相对稳定
资源消耗	大	小

预训练模型性能越强，微调或提示工程的效果越显著：预训练决定上限，微调决定落点。

依据 《大语言模型：训练、精调与对齐》表 5.2；第 5.1.1 节

上下文学习的适用性

GPT-3 提出的上下文学习（ICL）在输入中加入少量示例即可提升推理能力，不更新任何参数；但在部分任务上，用任务数据微调带来的提升要大得多。

表 5.3　GPT-3 使用少样本学习和微调结果比较

方法	MNLI-m（验证集，准确率 %）	RTE（验证集，准确率 %）
GPT-3 少样本学习	40.6	69.0
GPT-3 微调	89.5	85.4

+49.1

MNLI-m 准确率提升（百分点）

+16.4

RTE 准确率提升（百分点）

40.6 → 89.5

微调把“会用”变成“做得准”

- 即使少量任务数据，微调也能有效激活模型的任务解决能力。
- 微调后的模型仍可用于上下文学习，两者并不互斥。

依据 《大语言模型：训练、精调与对齐》表 5.3；第 5.1.1 节

微调与上下文学习的取舍

表 5.4　微调与上下文学习比较

特征	微调	上下文学习
参数更新	会更新模型参数	不更新模型参数
计算资源	需要 GPU 进行训练	只需推理资源
示例数量	通常需要几百到几千个样本	仅需少量示例（通常 < 10 个）
性能稳定性	更稳定	受示例质量影响大

任务数据充足

微调通常优于上下文学习：参数被真正改写，行为更稳定。

零样本场景

提示词工程更实用：不需要训练资源，改的是输入而不是参数。

> **类比**　预训练使模型获得广泛知识，好比阅读海量书籍；微调聚焦特定任务，好比针对性培训。预训练模型越强，微调与提示工程的效果越显著。

依据　《大语言模型：训练、精调与对齐》表 5.4；第 5.1.1 节

SFT 的损失计算

下游数据组织为问答对 $(\boldsymbol{x}, \boldsymbol{y})$：$\boldsymbol{x}$ 是输入（问题或指令），$\boldsymbol{y}$ 是期望输出（答案）。SFT 用交叉熵损失衡量模型在答案每个位置上的预测。

$$\mathcal{L}_{\mathrm{SFT}}(\boldsymbol{x}, \boldsymbol{y}) = -\sum_{i=0}^{|\boldsymbol{y}|} \log \Big(p_\theta(y_i \mid \boldsymbol{x}, \boldsymbol{y}_{<i}) \Big) \tag{5-1}$$

$p_\theta(y_i \mid \boldsymbol{x}, \boldsymbol{y}_{<i})$ 为给定输入与前缀时第 i 个答案词元的预测概率；求和只遍历答案 $\boldsymbol{y}$ 的位置，不遍历输入。

- 与预训练的差别不在损失形式，而在**条件**：SFT 建模 $p(\boldsymbol{y} \mid \boldsymbol{x})$，预训练建模 $p(\boldsymbol{x})$。
- 输入部分（系统提示、用户指令）不计损失，模型只需学会“看到这样的输入，应当续出怎样的答案”。

依据 《大语言模型：训练、精调与对齐》第 5.1.2 节（式 (5-1)）

SFT 与 KL 散度的关系

最小化交叉熵，等价于把模型预测分布 p_θ 拉向目标分布 q。

$$\mathcal{L}_{\mathrm{SFT}} = \mathbb{E}_{y_i \sim q}\big[-\log p_\theta(y_i | \boldsymbol{x}, \boldsymbol{y}_{<i}) \big] \tag{5-2}$$

$$\mathcal{L}_{\mathrm{SFT}} \propto D_{KL}\left(q \| p_\theta\right) \tag{5-3}$$

式 (5-2) 把求和写成对目标分布 q 的期望；式 (5-3) 说明两者只差与 p_θ 无关的常数项。

$$\mathcal{L}_{\mathrm{PT}} \propto D_{KL}(q_{\mathrm{data}} \| p_\theta) \tag{5-4}$$

q_{data} 为预训练语料的真实数据分布。SFT 与预训练的核心都是分布对齐，区别在数据与所建模的概率。

- 目标分布 q 为确定性分布（如独热分布）时，交叉熵与 KL 散度仅差常数项。
- SFT 本质上是把模型的条件分布 p_θ 向目标分布 q 对齐。

依据 《大语言模型：训练、精调与对齐》第 5.1.2 节（式 (5-2)–(5-4)）

SFT 与预训练的差别

$$\theta^{*} = \arg\min_{\theta} \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y})\sim\mathcal{D}} \mathcal{L}_{\mathrm{SFT}} \tag{5-5}$$

在任务数据集 $\mathcal{D}$ 上取期望，寻找使 SFT 损失最小的参数 θ^{*}。

区别一

数据分布

预训练用无标注通用文本；SFT 用特定任务的问答对。

区别二

条件建模

SFT 迁移条件概率 $p(\boldsymbol{y}|\boldsymbol{x})$；预训练建模联合概率 $p(\boldsymbol{x})$。

区别三

泛化目标

预训练聚焦语言表示能力；SFT 侧重指令跟随能力。

注 微调调整了模型的输出分布，使其更倾向于生成符合目标模式的序列；这种调整也可能带来性能退化或灾难性遗忘（见 5.1.4）。

SFT 训练样本的格式

语言模型的输入只能是文本，因此问答对要套上对话模板。模板一般含三种角色，由使用者自定义。

系统：你是一个智能助手。

用户：［翻译英文］ 大语言模型。

智能助手：Large Language Model <EOS>

问答模板的三种角色：系统声明行为准则，用户给出指令与输入，助手给出应生成的回复。

样本对

$(\boldsymbol{x}, \boldsymbol{y})$ = (“[翻译英文] 大语言模型”, “Large Language Model”)，“[翻译英文]”即指令。

常见封装

对话数据封装为消息列表，每条消息包含角色与内容，便于表示多轮交互。

- 训练时只计算助手回复部分（$\boldsymbol{y}$）的损失，系统提示与用户指令所在位置的损失被掩码忽略。
- 回复末尾追加终止标识符 <EOS>，让模型学会在生成完整回复后主动停止。

依据 《大语言模型：训练、精调与对齐》第 5.1.2 节（有监督微调数据示例）

OpenAI Messages 格式

同一段问答在工程实现中常写成 JSON 的 messages 列表：每项含 role 与 content，多轮对话就是按时间排列的消息数组。

```
[
  {"role": "system",
   "content": "你是一个智能助手。"},
  {"role": "user",
   "content": "[翻译英文] 大语言模型。"},
  {"role": "assistant",
   "content": "Large Language Model"}
]
```

多轮对话在数组末尾继续追加 user、assistant 成对的消息。

role 的三种取值

system 行为准则，只在开头一次；**user** 指令与输入；**assistant** 模型应生成的回复。

与对话模板的关系

messages 是**结构化表示**；送入模型前仍要按训练时的模板渲染成字符串。

终止符

自训模型把 <EOS> 显式写在 assistant 内容的末尾。

- 只对 `role: assistant` 的消息计损失——这就是掩码在消息层面的对应物。
- messages 与模板必须一一对应：训练用哪套渲染，推理就必须用同一套。

依据 《大语言模型：训练、精调与对齐》第 5.1.2 节（消息列表封装）

损失掩码与终止符

系统提示	用户指令与输入	助手回复 y	<EOS>
不计损失	不计损失	逐词元计损失	学会停止

终止标识

缺失 <EOS> 会怎样

训练数据缺失终止标识，或模型没学会输出它，推理时模型将无法正确停止生成，甚至出现重复输出。

模板一致性

训练与推理必须一致

训练阶段使用的对话模板，在测试与部署阶段必须严格保持一致，否则输入格式不匹配，性能会明显下降。

- 掩码的意义：让模型专注学习“给定上下文时如何作答”，而不是复述指令。
- 角色标签（系统 / 用户 / 助手）本身也是模板的一部分，换模板等于换了一种输入分布。

提示词微调

在输入数据中显式加入任务指令再训练模型，称为提示词微调（Prompt Tuning），也称硬提示词微调（Hard-Prompt Tuning）。加入的描述任务的文本，就是提示词。

同一翻译任务的三种指令模板

① 翻译中文句子到英文：{中文句子} → {英文句子}

② 中译英文：{中文句子} → {英文句子}

③ 中英翻译：{中文句子} 翻译为：{英文句子}

- 提示、输入与标签可以用灵活的结构组织。
- 不同提示词设计会显著影响模型在下游任务上的表现。
- 与软提示微调的差别：这里改的是**文本**，软提示改的是**可学习的连续向量**（见 5.2）。

依据 《大语言模型：训练、精调与对齐》第 5.1.3 节

灾难性遗忘

模型适应新任务时，参数被过度更新，预训练阶段习得的通用知识与能力严重流失——表现为相同输入下输出响应的概率分布发生系统性偏移。

表现一

安全能力退化

遗忘此前习得的安全对齐准则，生成有害内容的风险上升。

表现二

知识遗忘

预训练阶段积累的事实性知识在微调后难以稳定调用。

表现三

幻觉加剧

指令微调可能使模型更倾向于流畅作答，而忽略自身知识边界。

- 模型越大，SFT 所需的显存与计算资源急剧增加；特定下游任务上，在大模型上做 SFT 的成本常远高于在小模型上从头监督训练。
- 因此，**仅微调少量参数**来获得良好下游性能，成为关键问题——这是 5.2 节 PEFT 的动机。

灾难性遗忘的缓解策略

参数高效微调通过限制可更新参数的范围，降低优化难度、避免过度拟合，并在更新时更注重保留原始特征与知识。

通用 SFT 数据配比

在微调数据中混合部分通用任务数据，维持模型的基础能力。

改的是“喂什么”，让新旧分布同时出现。

预训练损失约束

在优化目标中加入预训练任务的损失，保留原始知识表示。

改的是“优化什么”，用损失项拉住旧能力。

> **要点** PEFT 在新任务学习过程中减少了对旧知识的干扰，更好地维持了模型的通用能力；它与上述两种策略互补，而不是替代关系。

下一节从“限制更新范围”这一思路出发，逐一介绍软提示、前缀、适配器与 LoRA 四类方法。

5.2

参数高效微调

冻结原模型，只训练少量新增参数：四类方法的差别在于“加在哪里”和“加什么”。

本节内容

5.2.1 定义

5.2.2 参数高效微调方法

参数高效微调（PEFT）

参数高效微调（PEFT）是一类方法的统称：只微调少量额外参数或模型的部分参数，显著节省计算与存储，同时力求接近全参微调的性能。

全参微调

对预训练模型的**所有**可学习参数进行更新。

效果上限高，但显存、存储与多任务切换成本都高。

参数高效微调

冻结主体参数，只更新少量新增或选定的参数。

性能接近全参微调，资源需求大幅下降。

- 传统的部分微调（冻结部分层、只微调其他层）也能减少参数量，但性能较为一般。
- 语言模型中典型的 PEFT 方法之一是**适配器**：在模型上附着额外的参数模块。
- 可训练参数大幅减少，训练时梯度与优化器（如 Adam）状态占用的显存也随之显著下降。

四类 PEFT 方法

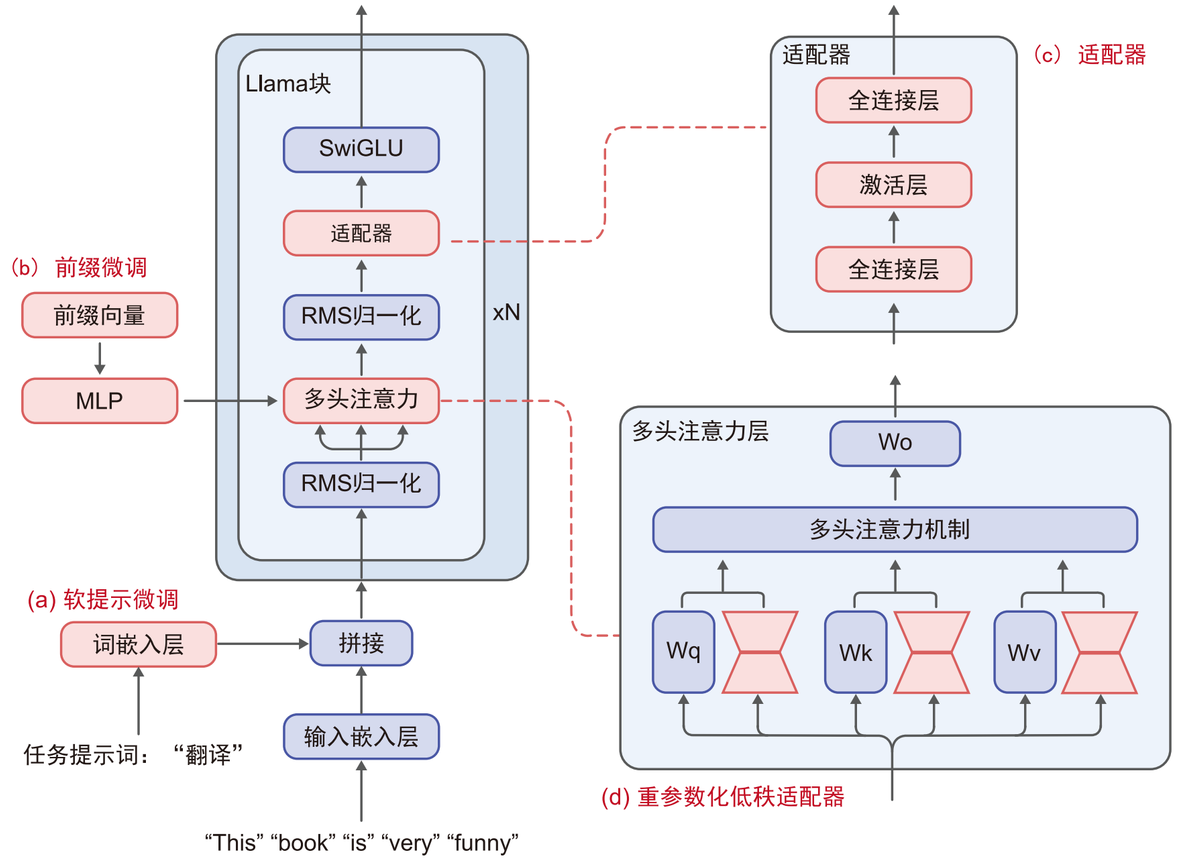


图 5.5　高效微调方法概览

(a) 软提示微调

输入序列前加可学习的任务提示向量。

(b) 前缀微调

可学习前缀向量，经 MLP 变换。

(c) 适配器

在层间插入小型多层感知机。

(d) 重参数

以 LoRA 为代表，在权重旁加低秩旁路。

四类 PEFT 方法的区别

方法	注入位置	可训练参数	训练时冻结
软提示微调	输入词嵌入序列之前	任务提示向量或软提示嵌入层	原模型全部参数
前缀微调	各层注意力的键、值之前	经 MLP 生成的前缀向量	原模型全部参数
适配器	模型层间	插入的多层感知机	原模型全部参数
LoRA	线性层旁路	低秩分解矩阵 $\boldsymbol{W}_A, \boldsymbol{W}_B$	原权重 $\boldsymbol{W}$

- 前三类都在**改变数据流**：要么在输入前拼接向量，要么在层间插入模块。
- LoRA 走的是**重参数化**：不改变网络结构，只给权重加一条可合并的旁路。
- LoRA 已衍生大量变体，其低秩思想还可扩展到梯度矩阵，如 GaLore。

依据 《大语言模型：训练、精调与对齐》第 5.2.2 节

软提示微调

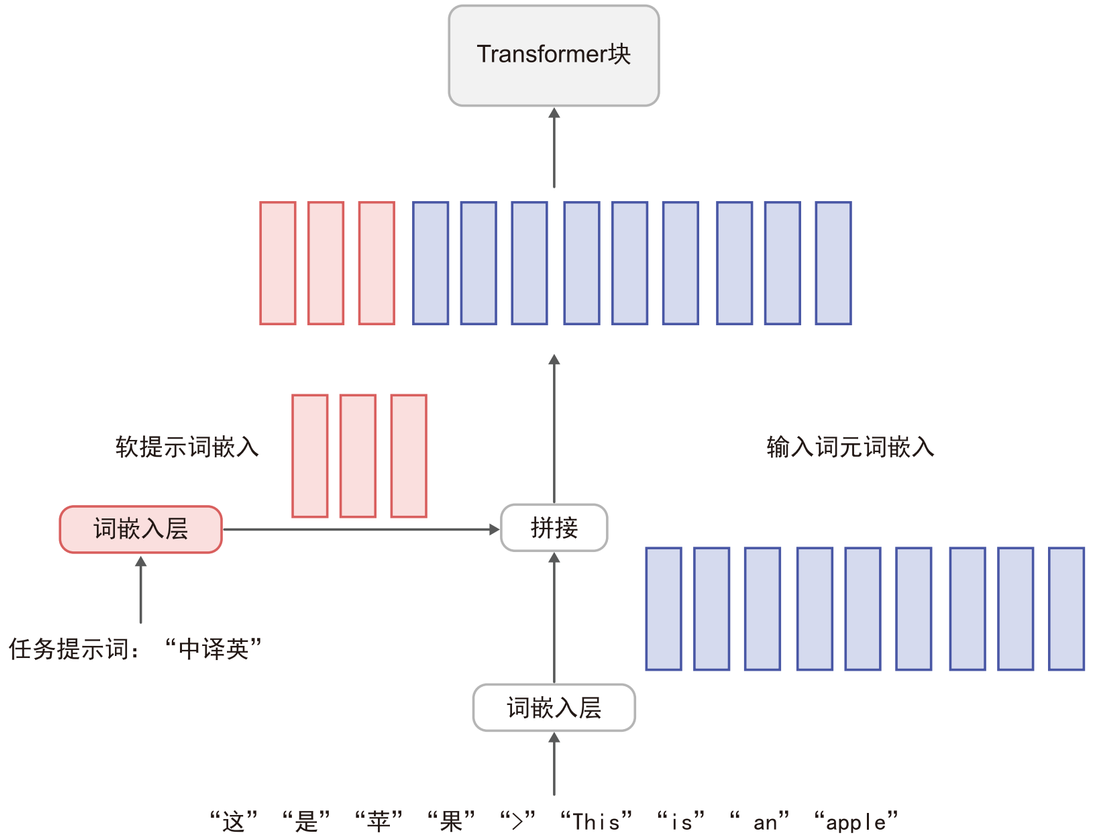


图 5.6　软提示词微调示意图

核心是引入一组可学习的连续向量（软提示），或一个专为软提示设计的可训练嵌入层；它们与输入文本的词嵌入序列拼接后送入模型。

- 预训练语言模型本身的参数全部冻结。
- 损失产生的梯度只用于更新软提示向量或软提示嵌入层。
- 它不是文本提示：这些向量没有对应的词，直接在嵌入空间中学习。

前缀微调

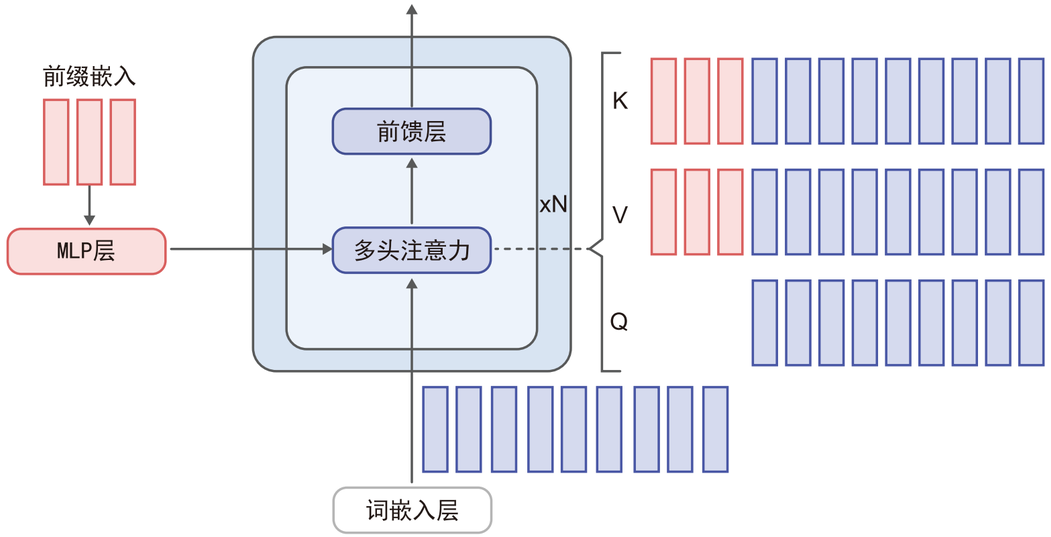


图 5.7　前缀微调示意图

同样在输入前加可学习的连续向量序列作为任务前缀，但前缀向量由多层感知机（MLP）变换得到，而不是直接使用可学习嵌入。

- MLP 的设计解决了直接拼接嵌入向量可能导致的训练不稳定问题，同时提升前缀向量的表达能力。
- 与软提示只在输入层加向量不同，前缀向量作用于 Transformer 的**每一层**：拼接在注意力的键与值序列之前。
- 仅需微调 GPT-2 约 0.1% 的参数，低数据量场景下常优于全参数微调，过拟合风险更小。

适配器微调

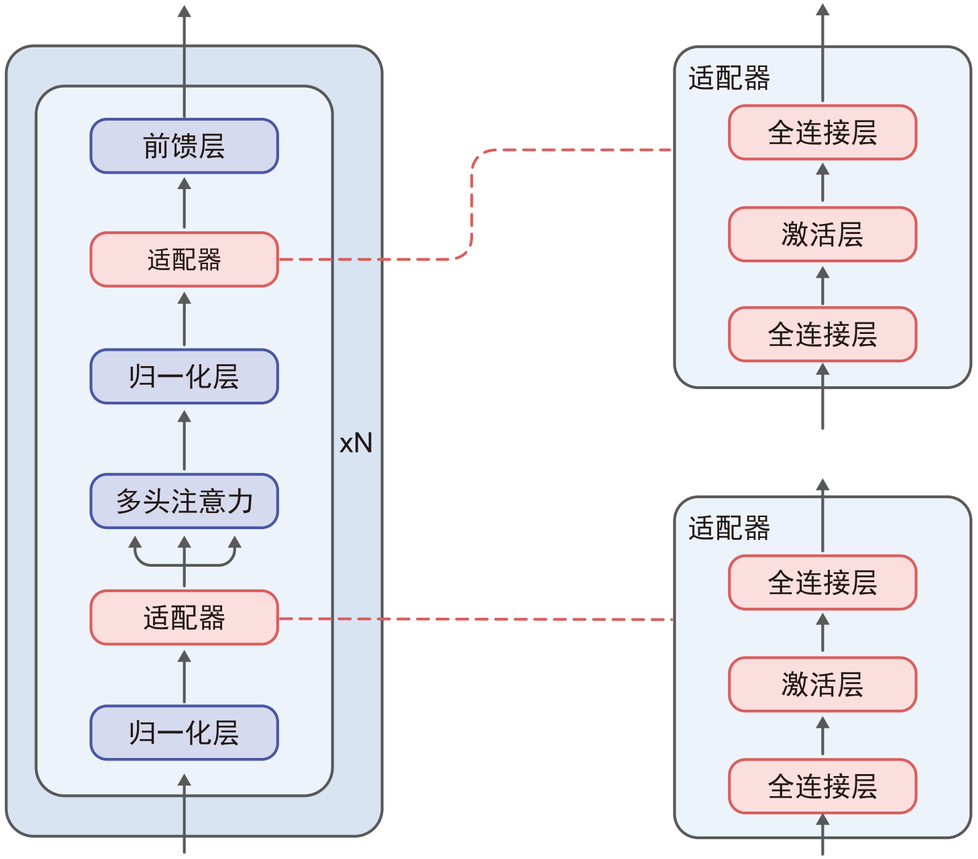


图 5.8　适配器微调方法示意图

在 Transformer 模型内部加入适配器层，适配器由多层感知机构成：全连接层、激活层，再全连接层。

- 适配器能覆盖网络中更多的特征学习，因此在多个下游任务上取得更好的性能表现。
- 代价是网络层变深，使用时增加预测时延——这一点在 5.3.4 的推理时延实验中会量化。
- 与软提示/前缀相比，适配器改的是**层内计算**，而不是输入或注意力序列。

LoRA 的结构

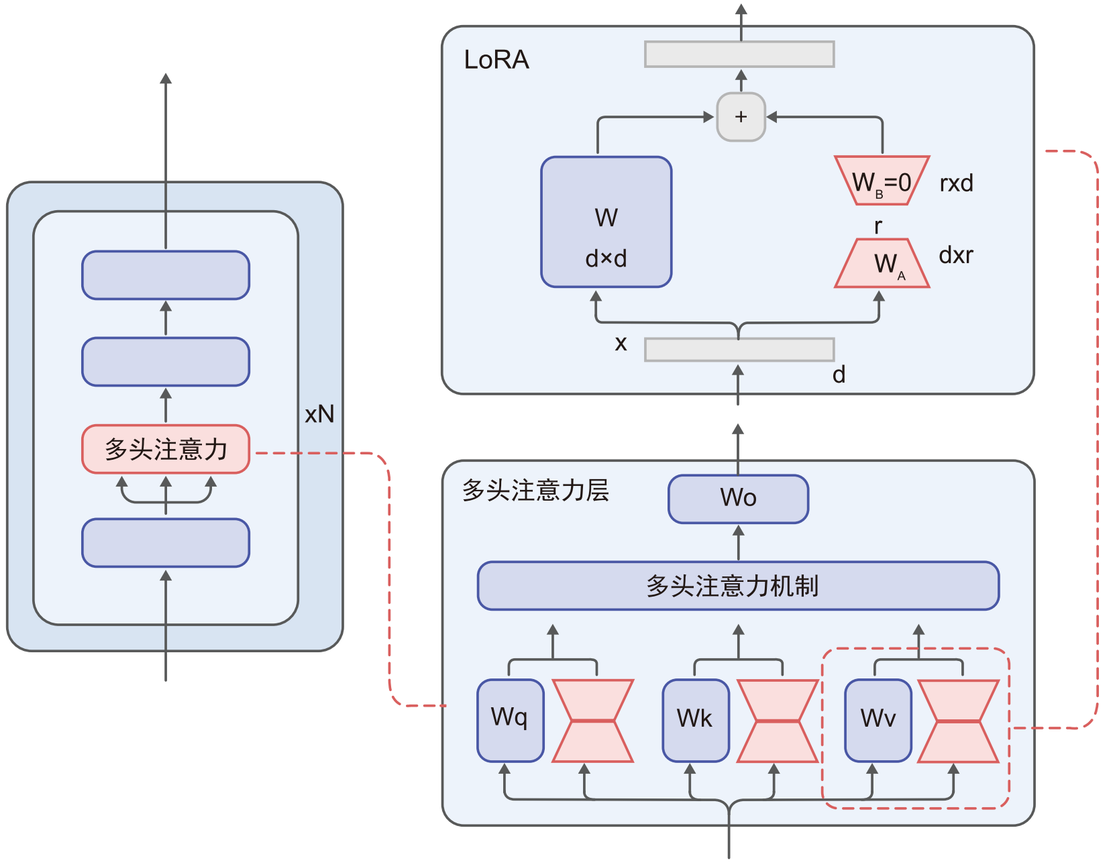


图 5.9　LoRA 微调方法示意图

LoRA 不改变原模型主体结构，只在线性层上增加旁路分支，仅调整旁路参数即可获得更好性能。

$$\boldsymbol{h} = \boldsymbol{x}\boldsymbol{W} + \boldsymbol{x}\boldsymbol{W}_A\boldsymbol{W}_B \tag{5-6}$$

$\boldsymbol{W} \in \mathbb{R}^{d\times d'}$，$\boldsymbol{W}_A \in \mathbb{R}^{d\times r}$，$\boldsymbol{W}_B \in \mathbb{R}^{r\times d'}$，$r$ 为秩。

- 旁路参数矩阵 $\boldsymbol{W}_A, \boldsymbol{W}_B$ 可在训练后并入原模型，推理时不引入额外延迟。
- 正因兼具精度与效率，LoRA 成为当前广泛使用的 PEFT 方法；原理在 5.3 节展开。

图 《大语言模型：训练、精调与对齐》图 5.9；式 (5-6)

LoRA 的权重合并

$$\boldsymbol{h} = \boldsymbol{x}\boldsymbol{W} + \boldsymbol{x}\boldsymbol{W}_A\boldsymbol{W}_B \tag{5-6}$$

训练时冻结 $\boldsymbol{W}$，只更新 $\boldsymbol{W}_A$ 与 $\boldsymbol{W}_B$；$\boldsymbol{x}\boldsymbol{W}$ 是原模型的输出，$\boldsymbol{x}\boldsymbol{W}_A\boldsymbol{W}_B$ 是旁路增量。

$$\boldsymbol{W}' \leftarrow \boldsymbol{W} + \boldsymbol{W}_A\boldsymbol{W}_B \tag{5-7}$$

训练完成后合并：$\boldsymbol{W}'$ 与原权重同形状，可直接替换，前向计算图与原模型完全相同。

训练时

两条分支并行：原权重路径冻结，旁路路径可训练。参数量由 $d \times d'$ 降为 $r(d + d')$。

推理时

合并为单个 $\boldsymbol{W}'$，只有一次矩阵乘法——这是 LoRA 相对适配器的关键优势。

式 (5-6)、(5-7) 是 5.3 节低秩重参数化推导的起点。

依据 《大语言模型：训练、精调与对齐》第 5.2.2 节（式 (5-6)、(5-7)）

5.3

LoRA 算法原理

本节内容

5.3.1 LoRA 定义

5.3.2 LoRA 分析

5.3.3 LoRA 微调参数量对比

5.3.4 性能分析

LoRA 假设微调的参数更新矩阵本身是低秩的：既然如此，就不必学习整个 $\Delta \boldsymbol{W}$，只需学习它的两个低秩因子。

LoRA 的提出动机

给定预训练自回归语言模型 $p_\theta(\boldsymbol{y} \mid \boldsymbol{x})$ 与下游数据集 $\mathcal{D} = \{(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)})\}_{i=1}^{N}$，全参数微调从 θ_0 出发学习更新量 $\Delta\theta$。

$$\max_{\theta} \sum_{(\boldsymbol{x},\boldsymbol{y})\in\mathcal{D}} \sum_{t=1}^{|\boldsymbol{y}|} \log\left(p_\theta(y_t|\boldsymbol{x}, \boldsymbol{y}_{<t})\right) \tag{5-8}$$

最大化对数似然，等价于最小化负对数似然；$\boldsymbol{x}$ 为输入上下文序列，$\boldsymbol{y}$ 为目标词元序列。

- 不同下游任务需要各自学习并存储一组**完整的** $\Delta\theta$。
- 模型规模极大时（如 GPT-3 的 175B 参数），为众多任务存储与部署这些 $\Delta\theta$，存储和切换成本极高。

思路 用一组规模小得多的参数 ϕ 表征 $\Delta\theta$，达到 $\phi \approx \Delta\theta$，且 $|\phi| \ll |\theta_0|$。

LoRA 的优化目标

$$\max_{\phi} \sum_{(\boldsymbol{x},\boldsymbol{y})\in\mathcal{D}} \sum_{t=1}^{|\boldsymbol{y}|} \log\left(p_{\theta_0+\phi}(y_t|\boldsymbol{x},\boldsymbol{y}_{<t})\right) \tag{5-9}$$

θ_0 冻结，只优化 ϕ；模型仍以 $\theta_0+\phi$ 的形式参与前向计算。

175B

GPT-3 的模型参数量 $|\theta_0|$

0.01%

LoRA 可训练参数量 $|\phi|$ 的下限

$\approx$

小参数近似完整参数更新的目标

- 关键不在“少训一点”，而在**把更新量约束到一个低维结构**里，使少量参数也能表达完整的任务适配。
- 换任务时只需替换 ϕ，原模型 θ_0 保持一份，存储与切换成本随之下降。

依据 《大语言模型：训练、精调与对齐》第 5.3.1 节（式 (5-9)）

LoRA 的低秩约束

预训练权重矩阵 $\boldsymbol{W}_0 \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ 通常满秩；LoRA 把它的更新量 $\Delta \boldsymbol{W}$ 约束为低秩分解形式。

$$\boldsymbol{h} = \boldsymbol{x}\boldsymbol{W}_0 + \boldsymbol{x}\Delta\boldsymbol{W} = \boldsymbol{x}\boldsymbol{W}_0 + \alpha \boldsymbol{x}\boldsymbol{W}_A\boldsymbol{W}_B \tag{5-10}$$

$\boldsymbol{W}_B \in \mathbb{R}^{r \times d_{\text{out}}}$，$\boldsymbol{W}_A \in \mathbb{R}^{d_{\text{in}} \times r}$，秩 $r \ll \min(d_{\text{in}}, d_{\text{out}})$；$\alpha$ 调节低秩更新项的相对权重。

第一步：前向

$\boldsymbol{x}\boldsymbol{W}_0$ 走冻结的原路径，$\alpha \boldsymbol{x}\boldsymbol{W}_A\boldsymbol{W}_B$ 走旁路；两条路径结果相加。

第二步：反向

梯度只回传到 $\boldsymbol{W}_A$ 与 $\boldsymbol{W}_B$，$\boldsymbol{W}_0$ 不更新；可训练参数量为 $r(d_{\text{in}} + d_{\text{out}})$。

- 低秩不是额外假设的“压缩技巧”，而是对“微调只改变少数方向”这一经验的参数化。

LoRA 的初始化

$\boldsymbol{W}_A$ 采用常规初始化，$\boldsymbol{W}_B$ 初始化为零矩阵，从而保证训练初期旁路 $\Delta \boldsymbol{W} = \mathbf{0}$，不改变原模型输出。

把 $\boldsymbol{W}_B$ 置零

$\Delta \boldsymbol{W} = \boldsymbol{W}_A \cdot \mathbf{0} = \mathbf{0}$，输出与原始模型完全一致。

梯度方面：$\boldsymbol{W}_A$ 的梯度含 $\boldsymbol{W}_B$ 因子，但 $\boldsymbol{W}_B$ 的梯度含 $\boldsymbol{W}_A$ 因子——$\boldsymbol{W}_A$ 非零，$\boldsymbol{W}_B$ 第一步就有梯度。

这是 LoRA 采用的方案。

把 $\boldsymbol{W}_A$ 置零

前向同样得到零增量，但 $\boldsymbol{W}_B$ 接收到的输入为 $\mathbf{0}$。

$\boldsymbol{W}_B$ 在这一步拿不到梯度，要等 $\boldsymbol{W}_A$ 被更新后才起步，起步更慢。

两种放法前向都是零增量；LoRA 选择把 $\boldsymbol{W}_B$ 置零，让两个因子从第一步起就同时更新。

要点 初始化决定了“从原模型出发”这一起点能否成立；零增量保证微调的第一步不破坏预训练能力。

反向推导：两个因子的梯度

把旁路写成 $\boldsymbol{s} = \boldsymbol{x}\boldsymbol{W}_A\boldsymbol{W}_B$，记上游梯度 $\boldsymbol{g} = \partial\mathcal{L}/\partial\boldsymbol{s}$。对两个低秩因子分别求导，看梯度里还剩什么。

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_B} = (\boldsymbol{x}\boldsymbol{W}_A)^T \boldsymbol{g}$$

$\boldsymbol{W}_B$ 的梯度以 $\boldsymbol{x}\boldsymbol{W}_A$ 为左因子——它只与 $\boldsymbol{W}_A$ 有关。

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_A} = \boldsymbol{x}^T \boldsymbol{g}\, \boldsymbol{W}_B^T$$

$\boldsymbol{W}_A$ 的梯度以 $\boldsymbol{W}_B^T$ 为右因子——它只与 $\boldsymbol{W}_B$ 有关。

> **命题** 两个因子的梯度互为另一方的线性函数：$\partial\mathcal{L}/\partial\boldsymbol{W}_B$ 以 $\boldsymbol{x}\boldsymbol{W}_A$ 为左因子，$\partial\mathcal{L}/\partial\boldsymbol{W}_A$ 以 $\boldsymbol{W}_B^T$ 为右因子。故两者都非零时每一步都有非零梯度；恰好一个为零时，另一个在该步梯度为零，而为零的那个立刻拿到梯度、离开零，只晚起步一步；只有两者**同时**为零，梯度才恒为零、旁路永不动，此时才真正停摆。

- **非零的一侧先动**：$\boldsymbol{W}_A \neq \boldsymbol{0}$ 时取 $\boldsymbol{x} = \boldsymbol{e}_i$（$i$ 为 $\boldsymbol{W}_A$ 的非零行），有 $\boldsymbol{x}\boldsymbol{W}_A \neq \boldsymbol{0}$，故 $\partial\mathcal{L}/\partial\boldsymbol{W}_B = (\boldsymbol{x}\boldsymbol{W}_A)^T\boldsymbol{g} \neq \boldsymbol{0}$；反过来同理。
- **零的一侧该步无梯度**：$\boldsymbol{W}_B = \boldsymbol{0}$ 时 $\partial\mathcal{L}/\partial\boldsymbol{W}_A = \boldsymbol{x}^T\boldsymbol{g}\cdot\boldsymbol{0} = \boldsymbol{0}$；$\boldsymbol{W}_A = \boldsymbol{0}$ 时 $\partial\mathcal{L}/\partial\boldsymbol{W}_B = \boldsymbol{0}$——这正是“两个都为零则永远为零”的来源。

零初始化放在哪个因子上

旁路 $\boldsymbol{x}\boldsymbol{W}_A\boldsymbol{W}_B$ 要让训练起步时不改变原模型，就必须在两个因子里放一个零矩阵——放哪一个，只看第 0 步的梯度落在谁身上。

零初始化放在 $\boldsymbol{W}_B$

第 0 步 $\boldsymbol{W}_A \neq \mathbf{0}$，故 $\partial\mathcal{L}/\partial\boldsymbol{W}_B = (\boldsymbol{x}\boldsymbol{W}_A)^T\boldsymbol{g} \neq \mathbf{0}$，$\boldsymbol{W}_B$ 立刻偏离零。

$\boldsymbol{W}_A$ 这一步梯度为零，第 1 步起两者同时更新。

LoRA 采用的方案。

零初始化放在 $\boldsymbol{W}_A$

第 0 步 $\partial\mathcal{L}/\partial\boldsymbol{W}_B = (\boldsymbol{x}\cdot\mathbf{0})^T\boldsymbol{g} = \mathbf{0}$，$\boldsymbol{W}_B$ 停一步。

但 $\boldsymbol{W}_A$ 的梯度 $\boldsymbol{x}^T\boldsymbol{g}\boldsymbol{W}_B^T$ 非零（$\boldsymbol{W}_B$ 是随机初值），$\boldsymbol{W}_A$ 离开零，第 1 步起两者同时更新。

能训，但晚起步一步。

- 两种放法前向都是零增量、都不至停摆：差别只在第 0 步梯度落在谁身上、谁要等一步。
- 只有 $\boldsymbol{W}_A$ 与 $\boldsymbol{W}_B$ **同时**为零，梯度才恒为零，旁路永远停在 $\mathbf{0}$——这才是真正的训练停摆。
- LoRA 选 $\boldsymbol{W}_B$：它的输入 $\boldsymbol{x}\boldsymbol{W}_A$ 从第一步就非零，两个因子不用等。

LoRA 与其他方法的对比

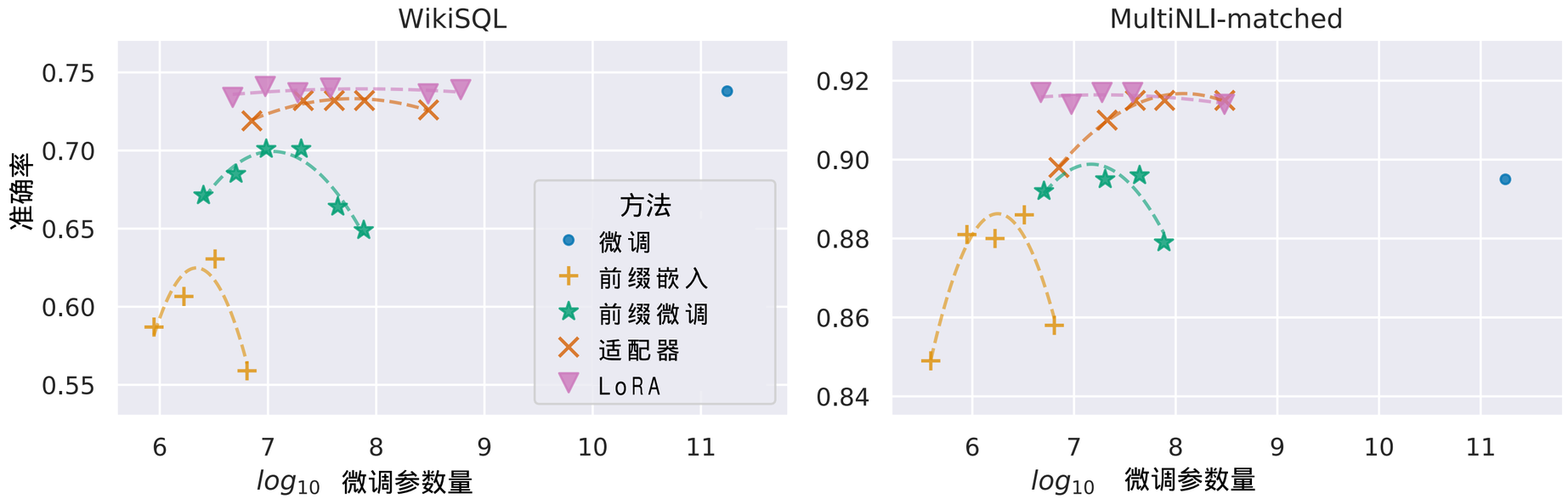


图 5.10　PEFT 方法微调效果对比（左：WikiSQL，右：MultiNLI）

读法

横轴为微调参数量，纵轴为任务准确率。

同等参数量

LoRA 优于适配器等其他方法。

变化平缓

参数增加时准确率起伏小，反映更新矩阵的低秩特性。

从优化角度看，LoRA 相当于在全参数空间的一个低秩子空间内搜索最优解，因此性能通常略低于全参数微调。

LoRA 的适配矩阵选择

在 GPT-3 上为不同类型的注意力权重（$\boldsymbol{W}_q, \boldsymbol{W}_k, \boldsymbol{W}_v, \boldsymbol{W}_o$）应用不同秩的 LoRA，比较下游性能。

表 5.5　注意力层各投影层进行 LoRA 微调性能对比表

权重类型	$\boldsymbol{W}_q$	$\boldsymbol{W}_k$	$\boldsymbol{W}_v$	$\boldsymbol{W}_o$	$\boldsymbol{W}_q, \boldsymbol{W}_k$	$\boldsymbol{W}_q, \boldsymbol{W}_v$	四者
秩 r	8	8	8	8	4	4	2
WikiSQL（±0.5%）	70.4	70.0	73.0	73.2	71.4	**73.7**	**73.7**
MultiNLI（±0.1%）	91.0	90.8	91.0	91.3	91.3	91.3	**91.7**

- 各方案的可训练参数量同为 18M，差别只在“把钱花在哪个矩阵、用多大秩”。
- 较小的秩 r 同时适配**更多**权重矩阵，通常优于大秩适配少数矩阵：MultiNLI 上 $r = 2$ 适配四个矩阵达到 91.7%。

依据 《大语言模型：训练、精调与对齐》表 5.5；第 5.3.2 节

LoRA 的秩选择

固定权重类型，只改秩 r：$r = 2$ 时 LoRA 已达较好性能，继续增大 r 并不能显著提升表现。

表 5.6　秩大小与 MultiNLI 测试性能表

权重类型	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
$\boldsymbol{W}_q$	90.7	90.9	91.1	90.7	90.7
$\boldsymbol{W}_q, \boldsymbol{W}_v$	91.3	91.4	91.3	91.6	91.4
四者	91.2	91.7	91.7	91.5	91.4

- 增加秩，不等于引入更多能有效提升下游性能的特征子空间——与图 5.10 的现象一致。
- 即使把秩降到 $r = 1$，性能也只轻微下降；这些数据集上 $r = 1$ 已足够适配 $\boldsymbol{W}_q$ 与 $\boldsymbol{W}_v$。
- 单独训练 $\boldsymbol{W}_q$ 则需要更大的 r：只覆盖一个矩阵时，秩成为表达能力的瓶颈。

依据 《大语言模型：训练、精调与对齐》表 5.6；第 5.3.2 节

不同秩学到的子空间

取同一预训练模型上 $r=8$ 与 $r=64$ 的适配矩阵 $\boldsymbol{A}_{r=8}\in\mathbb{R}^{d\times 8}$、$\boldsymbol{A}_{r=64}\in\mathbb{R}^{d\times 64}$，各自做 SVD，取前 k 个左奇异向量张成子空间。

$$\phi_{\text{grassmann}}(\boldsymbol{A}_{r=8},\boldsymbol{A}_{r=64},i,j)=\frac{\|\boldsymbol{U}^{(i)}_{\boldsymbol{A}_{r=8}}\boldsymbol{U}^{(j)T}_{\boldsymbol{A}_{r=64}}\|_F^2}{\min(i,j)}\in[0,1] \tag{5-11}$$

$\boldsymbol{U}^{(k)}\in\mathbb{R}^{k\times d}$ 为前 k 个行奇异向量构成的矩阵，$\boldsymbol{U}^{(i)}_{\boldsymbol{A}_{r=8}}\boldsymbol{U}^{(j)T}_{\boldsymbol{A}_{r=64}}\in\mathbb{R}^{i\times j}$。值为 1 表示子空间完全重合，为 0 表示独立。

- 分子是两个子空间投影矩阵之积的 Frobenius 范数平方，分母 $\min(i,j)$ 把结果归一化到 $[0,1]$。
- 对 $\boldsymbol{W}_B$ 的 SVD 右奇异矩阵可做同样的分析。

依据 《大语言模型：训练、精调与对齐》第 5.3.2 节（式 (5-11)）

主奇异方向的一致性

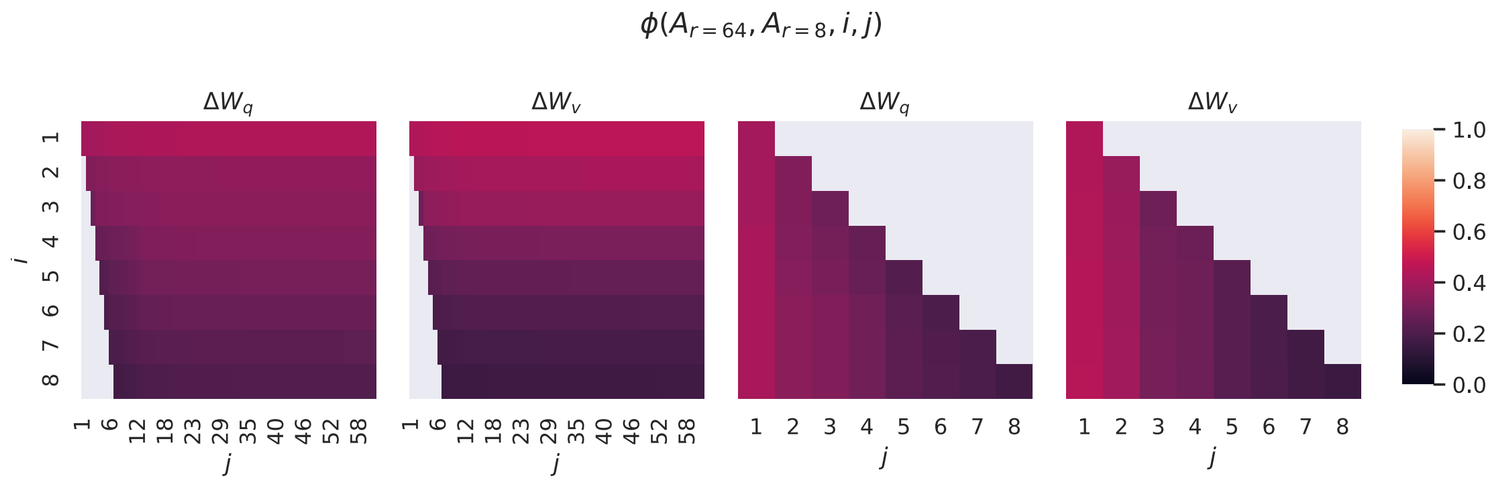


图 5.11　低秩子空间相似性分析

- GPT-3 175B 第 48 层，$\Delta \boldsymbol{W}_q$ 与 $\Delta \boldsymbol{W}_v$ 的子空间相似性。
- (a) 与 (c) 可拼成 $i \times j$ 热力图，即 $\boldsymbol{U}_{\boldsymbol{A}_{r=8}}^{(i)} \boldsymbol{U}_{\boldsymbol{A}_{r=64}}^{(j)T}$ 的可视化。
- $i = j = 1$ 时相似度仍大于 0.5，最低秩方向高度一致。

解释　这正解释了表 5.6 中 $r = 1$ 仍能有效微调：即使秩极小，LoRA 仍能捕获最关键的参数更新方向。

ΔW 与 W 的关系

把 ΔW 的左右奇异向量作为投影基，投影原权重 W，看能保留多少能量：$\|U_{\Delta W} W V_{\Delta W}^T\|_F$。

投影

SVD 取 ΔW 前 r 维左右奇异向量 $U_{\Delta W} \in \mathbb{R}^{r \times d_{\text{in}}}$、$V_{\Delta W} \in \mathbb{R}^{r \times d_{\text{out}}}$，计算 $U_{\Delta W} W V_{\Delta W}^T \in \mathbb{R}^{r \times r}$。

对照组

分别用 W 自身前 r 个奇异向量、随机矩阵 R 的奇异向量构成投影基 U_*, V_*，同样计算范数作比较。

若 ΔW 的方向与 W 的重要方向无关，则其投影范数应接近随机基的结果；实验给出的数值见下一页表 5.7。

- 比较基准是 $\|W_q\|_F = 61.95$，以及 $\|\Delta W_q\|_F$ 在 $r = 4$ 时为 6.91、$r = 64$ 时为 3.57。

依据 《大语言模型：训练、精调与对齐》第 5.3.2 节

ΔW 的投影范数

表 5.7　矩阵 $U_*^T W_q V_*^T$ 的 Frobenius 范数计算结果

投影基	$r=4$			$r=64$		
	ΔW_q	W_q	R	ΔW_q	W_q	R
$\|U_* W_q V_*^T\|_F$	0.32	21.67	0.02	1.90	37.71	0.33

- **与随机基比**：ΔW_q 基得到 0.32 / 1.90，远大于随机基的 0.02 / 0.33，说明 ΔW 与 W 存在较强的方向关联。
- **能量集中度**：$\|\Delta W_q\|_F / \|U_{\Delta W} W_q V_{\Delta W}^T\|_F$ 在 $r=4$ 时约 21.59，$r=64$ 时约 1.88——少数核心方向上 W 的能量高度集中。
- **与自身基比**：ΔW_q 基的结果（0.32 / 1.90）远小于 W_q 自身基（21.67 / 37.71），说明它也捕获了原矩阵中相对次要的方向。

结论：LoRA 的核心变化方向与 W 的关键方向高度重合，同时强化了预训练未被充分优化的方向。

依据 《大语言模型：训练、精调与对齐》表 5.7；第 5.3.2 节

LoRA 的参数量

设词嵌入维度为 d_{model}、秩为 r。对单一线性层，LoRA 引入的参数量是两个低秩矩阵之和。

$$|\Theta| = 2 \times d_{\text{model}} \times r \tag{5-12}$$

$\boldsymbol{W}_A$ 为 $d_{\text{model}} \times r$，$\boldsymbol{W}_B$ 为 $r \times d_{\text{model}}$，两者参数量相同，故乘以 2。

$$\rho = \frac{\Theta_{\text{LoRA}}}{\Theta_{\text{FT}}} = \frac{2r}{d_{\text{model}}} \tag{5-13}$$

全参微调单个线性层为 $\Theta_{\text{FT}} = d_{\text{model}} \times d_{\text{model}}$，相除后 d_{model} 约去，只剩 $2r/d_{\text{model}}$。

4

典型秩 r

4096

典型 d_{model}

0.195%

LoRA 需微调的参数量占比

$r \ll d_{\text{model}}$ 时 ρ 很小：微调成本几乎只与秩成正比，与模型宽度成反比。

LoRA 的推理时延

$\boldsymbol{W} \leftarrow \boldsymbol{W} + \boldsymbol{AB}$ 合并后，微调模型与原模型的计算图相同，因此推理时只多一次矩阵乘法的差别可以忽略。

表 5.8　推理时延实验结果（GPT-2 Medium，NVIDIA Quadro RTX8000，单次前向延迟 ms，100 次试验取平均；$|\Theta|$ 为适配器层可训练参数量）

批次数量 / 序列长度	32 / 512	16 / 256	1 / 128
$\lvert\Theta\rvert$	0.5M	11M	11M
LoRA 微调	1449.4±0.8	338.0±0.6	19.8±2.7
$\text{Adapter}^{\text{L}}$	1482.0±1.0（+2.2%）	354.8±0.5（+5.0%）	23.9±2.1（+20.7%）
$\text{Adapter}^{\text{H}}$	1492.2±1.0（+3.0%）	366.3±0.5（+8.4%）	25.8±2.2（+30.3%）

- 在线短序列场景（批次 1、长度 128）中，适配器层引入的推理延迟相当显著：最高 +30.3%。
- LoRA 因可合并，额外推理延迟可忽略不计；批次与序列越长，三种方法的差距越小。

依据 《大语言模型：训练、精调与对齐》表 5.8；第 5.3.4 节

LoRA 在下游任务上的效果

表 5.9　LoRA 微调在下游任务性能表现

模型（方法）	训练参数量	WikiSQL 准确率	MNLI-m 准确率	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 ($\text{Adapter}^{\text{H}}$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 ($\text{Adapter}^{\text{H}}$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

- LoRA 仅微调 0.01% 的参数即可达到与全参微调相当的精度，显著优于其他参数高效方法。
- 波动范围：WikiSQL 约 ±0.5%，MNLI-m 约 ±0.1%，SAMSum 三项分别约 ±0.2%、±0.2%、±0.1%。

依据 《大语言模型：训练、精调与对齐》第 5.3.4 节；表 5.9

5.4

QLoRA

LoRA 省下了优化器状态与梯度，但基座模型本身仍要按 16-bit 装进显存。QLoRA 解决的是后者。

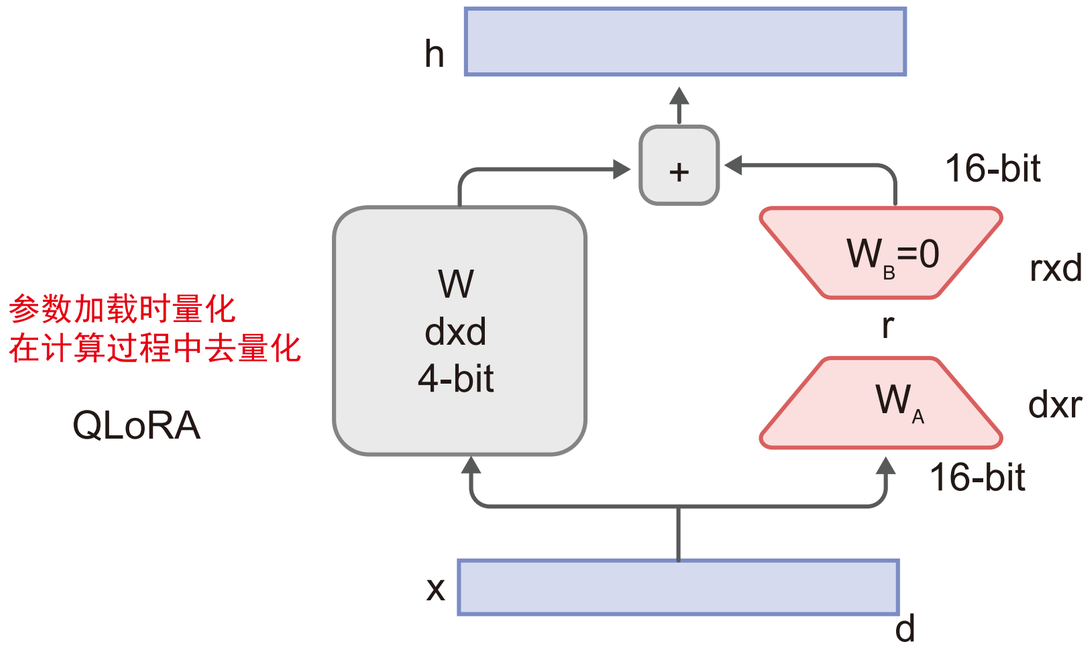
本节内容

量化基座 + 低秩适配

NF4 与显存账本

消费级 GPU 上的 65B 微调

QLoRA 的方法



在标准 LoRA 框架上，冻结预训练模型参数、仅更新低秩适配器，同时对基座参数做 4-bit NormalFloat（NF4）量化。

- 量化显著降低**加载模型参数**所需的显存开销。
- 适配器仍以较高精度训练，反传时按需反量化。
- 理论上可在一张 48GB 显存的 GPU 上微调 65B 参数的模型。

效果

与 16-bit 全参微调接近；Guanaco 达 ChatGPT 99.3% 的水平，单卡微调 24 小时。

显存账本

Llama-2-7B 的 FP16 参数约 14GB，NF4 后加载开销约 4GB；峰值还要加优化器状态与激活值。

图 《大语言模型：训练、精调与对齐》图 5.12

QLoRA 的显存节省

一次微调的显存由几块组成：基座参数、梯度、优化器状态、激活值。LoRA 与 QLoRA 各自压缩其中不同的部分。

全参微调

四块全都要

参数、梯度、Adam 状态都要按全量保存，显存需求最高。

LoRA

砍掉梯度与优化器

基座冻结，只有少量适配器参数需要梯度与优化器状态。

QLoRA

再把基座压到 4-bit

参数存储本身从 16-bit 降到 4-bit，加载开销约为原来的四分之一。

- 两者叠加后，峰值显存主要由“量化基座 + 少量适配器状态 + 激活值”构成。
- 这正是 5.5 节实验中 QLoRA 峰值显存最低、但吞吐略降的原因：反量化需要额外计算。

依据 《大语言模型：训练、精调与对齐》第 5.4 节

5.5

微调方法对比

同一台机器、同一个模型，比较三种方法：训练参数、存储占用、吞吐与困惑度。

本节内容

实验设置

效率与效果对比

如何选择

QLoRA 的实验设置

在 PubMed 数据集上取 400K 词元做预训练实验，统一使用 Llama-2-7B 与同一机器平台，对比全参微调、LoRA 与 QLoRA。

效率指标

训练要花多少

训练时的峰值显存占用（GB）；训练吞吐量（Tokens/s，每秒训练词元数）。

效果指标

训得好不好

困惑度（PPL）：模型对文本的预测难度，越低越好。

- 三个变量的关系值得注意：显存低不等于吞吐高，QLoRA 就是典型。
- 基线（未微调）的 PPL 为 7.53，三种方法都能把它降到 6 以下。

QLoRA 的效率与效果

表 5.10　全参微调、LoRA 和 QLoRA 三种方法的训练效率对比

方法	训练参数	存储占用（GB）	吞吐（Tokens·s^{-1}）	PPL
基线	/	/	/	7.53
全参微调	6.74B	38.72	1334.72	5.56
LoRA	0.32B	16.32	**1954.07**	**5.81**
QLoRA	0.32B	**7.52**	1579.16	5.91

精度效果

全参 5.56，LoRA 5.81，QLoRA 5.91。全参最好，两个 PEFT 方法差距很小。

训练吞吐

LoRA 最高 1954.07 Tokens/s；QLoRA 因反量化降至 1579.16。

峰值显存

全参 38.72GB 最高；QLoRA 仅 7.52GB，约为全参的 1/5.15。

全参微调的峰值显存是 QLoRA 的约 5.1 倍（38.72 / 7.52 ≈ 5.15）。

依据 《大语言模型：训练、精调与对齐》表 5.10；第 5.5 节

三种微调方法的取舍

显存充足、追求精度

全参微调：PPL 最低（5.56），但需要 38.72GB 峰值显存与 6.74B 训练参数。

追求训练速度

LoRA：吞吐最高（1954.07 Tokens/s），精度接近全参（5.81），显存居中（16.32GB）。

显存紧张

QLoRA：峰值仅 7.52GB，PPL 5.91 仍明显优于基线 7.53；代价是反量化带来的吞吐下降。

消费级 GPU

QLoRA 特别适合在显存有限的消费级显卡（24GB 或以下）上微调大型模型。

> **读法** 不要只看 PPL 排名：三种方法的 PPL 差异不到 0.4，而显存相差 5 倍、吞吐相差近 1.5 倍。选择取决于硬件约束。

5.6

小结

微调把预训练能力导向任务，PEFT 把这件事的成本降到可承受；LoRA 在精度与效率之间找到了一个通用解。

本节内容

方法与线索回顾

LoRA 的扩展方向

延伸阅读

本章小结

全参微调
成本高、存储多

PEFT
只更新 <1% 参数

提示 / 前缀 / 适配器
改数据流

LoRA
改权重旁路

QLoRA
基座 4-bit 量化

- 全量参数微调计算成本高昂；PEFT 仅更新一小部分参数（通常少于原模型的 1%），即可达到媲美全参微调的效果。
- 代表性 PEFT 方法包括提示微调、前缀微调、适配器与 LoRA。
- LoRA 因高效与灵活被广泛采用，并可针对不同场景扩展，例如 QLoRA 把基座量化到 4-bit。

LoRA 的扩展方向

路线一

改进初始化

LoRA-GA 优化 LoRA 的参数初始化，使低秩旁路从一开始就对准有用的更新方向。

路线二

改进梯度策略

GaLore 把低秩思想用于梯度矩阵，降低训练时的优化器显存；LoRA-PRO 则改进低秩适配器本身的性质。

路线三

系统梳理变体

LoRA 已有大量变体，可阅读综述以获得更全面的图景。

回看 5.3 节的三个实验分别回答了三个问题：适配哪些矩阵（表 5.5）、秩取多大（表 5.6）、子空间是否重合（图 5.11）。它们共同支撑“低秩假设”这一理论基础。

应用现状 LoRA 当前系列技术在科研或实际生产中应用较少，原因在于追求更高的模型训练精度；相较之下，LoRA 在扩散生成领域有更广泛的应用。

思考题与实践题

1 式 (5-3) 说明 SFT 等价于最小化 KL 散度。若目标分布 q 不是独热分布，这一等价还成立吗？

2 在 OpenAI Messages 格式下如何做掩码？请说明 role 序列与 label 的对应关系，并写出多轮对话中每条消息应取的值。

3 软提示微调与提示词微调都叫“提示”，二者在参数空间中的差别是什么？

4 前缀微调作用于每层的键和值，软提示只作用于输入层。这一差别为何带来表达能力差异？

5 LoRA 为何把 $\boldsymbol{W}_B$ 而非 $\boldsymbol{W}_A$ 零初始化？请从前向与反向两个角度说明。

6 表 5.6 中 r 从 1 增到 64，性能基本不变。结合式 (5-11) 的 Grassmann 距离解释。

7 实践：实现一个 LoRA 层——含 $\boldsymbol{W}_A, \boldsymbol{W}_B$、零初始化、$\alpha/r$ 缩放与 `merge()`，并验证合并前后输出一致。

8 实践：收集 10 万条 SFT 数据，分别做全参训练与 LoRA 微调，在同一 MMLU 子集上对比基准表现。